CSE 4616 – Wireless Networks

Tutorial: Introduction to OMNeT++ and INET

Your First Network Simulation

Tools: OMNeT++ IDE — INET Framework

Contents

1. What Is OMNeT++? Understanding the Big Picture

Before writing a single line of code, it is important to understand *why* we use OMNeT++ and what problem it solves.

Imagine you want to study how data packets travel across a wireless network with 50 devices. You cannot afford to buy 50 real computers and routers, set them up in a lab, and run experiments. Instead, you build a **simulation** — a software model of the network that behaves like the real thing — and run experiments on your laptop.

What is OMNeT++?

OMNeT++ (Objective Modular Network Testbed in C++) is a **discrete-event simulation framework**. It lets you describe a network of communicating modules and simulate how events (packet arrivals, transmissions, timeouts) unfold over time — without needing real hardware.

Think of it as a **flight simulator for networks**: pilots train on simulators before flying real planes; network researchers experiment on OMNeT++ before deploying real protocols.

1.1 What Does "Discrete-Event" Mean?

In a discrete-event simulation, time jumps from one **event** to the next. Nothing happens between events. For example:

- At $t = 1.000\,\text{s}$: Host A sends a packet.
- At $t = 1.003\,\text{s}$: The packet arrives at the router.
- At $t = 1.005\,\text{s}$: The router forwards the packet to Host B.

The simulation clock skips directly from one event timestamp to the next. This makes it extremely fast even for large networks.

1.2 What Is INET?

OMNeT++ on its own is just a simulation engine — it provides no networking protocols. **INET** is a free, open-source library of pre-built network components that runs on top of OMNeT++.

What is INET?

INET provides ready-made, realistic implementations of: Ethernet, Wi-Fi (IEEE 802.11), IP, TCP, UDP, routing protocols, and many application types. Instead of coding a UDP sender from scratch, you simply use INET's `UdpBasicApp` module.

Think of OMNeT++ as the **game engine** and INET as the **game content** (the actual network devices and protocols).

1.3 The Two Files You Always Need

Every OMNeT++/INET simulation requires exactly two types of files:

File	What it describes	Real-world analogy
`.ned`	The network topology: which devices exist and how they are connected	An architect's floor plan of a building
`.ini`	The simulation configuration: how devices behave, traffic parameters, run duration	An operations manual telling occupants what to do inside the building

Table 1: The two core files in every OMNeT++ simulation

You will spend most of your time writing and editing these two files.

2. Installing OMNeT++ and INET

2.1 Installing OMNeT++

1. Go to `https://omnetpp.org/download/` and download the installer for your operating system (Windows, Linux, or macOS).
2. Extract the downloaded archive to a folder with **no spaces in the path** (e.g., `C:\omnetpp` on Windows, or `/home/yourname/omnetpp` on Linux).
3. Follow the installation guide in the official documentation. On Linux, run `./configure` then `make` inside the extracted folder.
4. Launch the OMNeT++ IDE by running `omnetpp` (Linux/macOS) or double-clicking the IDE shortcut (Windows).

Common Mistake

Never install OMNeT++ in a folder path that contains spaces (e.g., `C:\Program Files\omnetpp`). This causes build errors that are very difficult to diagnose.

2.2 Importing and Building INET

1. In the OMNeT++ IDE, go to **File** → **Import** → **General** → **Existing Projects into Workspace**.
2. Browse to the INET folder (downloaded separately from `https://inet.omnetpp.org`) and select it.
3. Click **Finish**. INET will appear in the Project Explorer.
4. Right-click on the **inet** project → **Build Project**. This may take several minutes the first time.
5. When the build completes with no errors, INET is ready to use.

Tip

If you see red error markers after building, try right-clicking the project and selecting **Refresh**, then build again. If errors persist, check that your project references INET via **Project** → **Properties** → **Project References**.

3. The OMNeT++ IDE: A Quick Tour

When you open the OMNeT++ IDE for the first time, you will see several panels. Understanding them from the start saves a lot of confusion.

3.1 Project Explorer (Left Panel)

This is your **file browser**. All your projects, NED files, INI files, and result folders appear here. You navigate to files by expanding folders, just like Windows Explorer or macOS Finder.

3.2 Editor Area (Centre Panel)

This is where you write and edit your NED and INI files. The IDE provides **syntax highlighting** (keywords appear in colour) and **auto-complete** (press `Ctrl+Space` to see suggestions).

The NED editor also has a **graphical view**: click the canvas tab at the bottom of the editor to see a visual diagram of your network instead of raw text. You can switch between text and graphical views freely.

3.3 Console / Problems Panel (Bottom Panel)

- **Problems tab:** Shows compile-time errors in your NED files (red markers). Always fix these before running.
- **Console tab:** Shows output printed by the simulation at runtime (e.g., ping results, log messages).

3.4 Qtenv: The Simulation Window

When you run a simulation, a separate window called **Qtenv** opens. This is the graphical runtime environment where you can:

- See your network drawn on a canvas with animated packets flying between nodes.
- Press **Play** to run continuously, **Step** to advance one event at a time, or **Pause** to stop and inspect.
- Click on any module (host, router, etc.) to inspect its internal state, routing table, or application parameters.
- Use **Simulate** → **Run Until...** to stop at a specific simulated time.

4. Understanding NED Files

A NED (**N**etwork **E**escription) file is where you define the **structure** of your network. Think of it as the answer to the question: *"What devices exist in my network, and which cables connect them?"*

4.1 The Overall Structure of a NED File

Every NED file follows this skeleton:

```
// Optional comments
package your.package.name;    // Where this file lives

import ModuleType1;           // Bring in pre-built modules
import ModuleType2;

network YourNetworkName
{
    submodules:
        // declare your devices here

    connections:
        // wire them together here
}
```

Listing 1: Skeleton of a NED file

Each part has a specific role, explained in the sections below.

4.2 Package Declaration

```
package inet.examples.wireless.wiredandwirelesshostswithap;
```

Listing 2: Package declaration

The `package` line tells OMNeT++ **where this file lives** inside the project folder tree. It must exactly mirror the folder path. If your file is at:

`inet/examples/wireless/wiredandwirelesshostswithap/MyNetwork.ned`

then the package must be:

`inet.examples.wireless.wiredandwirelesshostswithap`

Notice that folder separators (`/`) become dots (`.`).

Common Mistake

If the package name does not match the actual folder path, OMNeT++ will fail to find your network when you try to run it. Always double-check this.

4.3 Import Statements

```
import inet.networklayer.configurator.ipv4.Ipv4NetworkConfigurator
    ;
import inet.node.ethernet.Eth100M;
import inet.node.inet.StandardHost;
import inet.node.inet.WirelessHost;
import inet.node.wireless.AccessPoint;
import inet.physicallayer.wireless.ieee80211.packetlevel.
    Ieee80211ScalarRadioMedium;
```

Listing 3: Import statements bring INET modules into scope

Import statements work exactly like `import` in Python or `#include` in C. They tell OMNeT++ which pre-built INET module types you want to use. Without importing a type, you cannot use it as a submodule.

The full list of available INET modules can be browsed in the INET documentation at https://inet.omnetpp.org/docs/.

4.4 The Network Block

```
network WiredAndWirelessHostsWithAP
{
    submodules:
        // ...
    connections:
        // ...
}
```

Listing 4: Network block declaration

The `network` keyword declares a **compound module** that represents your entire simulated network. The name you give here (`WiredAndWirelessHostsWithAP`) must be referenced in the `omnetpp.ini` file via `network = WiredAndWirelessHostsWithAP`.

4.5 Submodules: Declaring Your Devices

Submodules are the **devices** inside your network. Each submodule has:

- A **name** (what you call it, e.g., `wiredHost1`)
- A **type** (what kind of device it is, e.g., `StandardHost`)
- Optional **parameters** (configuration specific to that device)

```
submodules:
    // A wired host (regular PC with Ethernet)
    wiredHost1: StandardHost {
        parameters:
            @display("p=296,134");   // position on canvas (x,y)
    }

    // A wireless host (laptop/phone with Wi-Fi)
```

```
    wirelessHost1: WirelessHost {
        parameters:
            @display("p=62,69");
    }

    // An access point (Wi-Fi router)
    accessPoint: AccessPoint {
        parameters:
            @display("p=205,69");
    }

    // The IP configurator (assigns addresses automatically)
    configurator: Ipv4NetworkConfigurator {
        parameters:
            assignDisjunctSubnetAddresses = false;
            @display("p=100,100;is=s");
    }

    // The wireless radio medium (simulated "air")
    radioMedium: Ieee80211ScalarRadioMedium {
        parameters:
            @display("p=100,200;is=s");
    }
```

Listing 5: Declaring submodules

What is `@display`?

`@display("p=x,y")` sets the visual position of the module on the Qtenv canvas. The values are pixel coordinates. `is=s` makes the icon small. These annotations have **no effect on simulation logic** — they only affect how the network looks visually.

Why do we need `radioMedium`?

In real life, Wi-Fi signals travel through the air. In OMNeT++, we must explicitly create this "air" as a module called `radioMedium`. Every wireless device in the network automatically uses it. Without it, wireless communication is impossible — the simulation would run but no Wi-Fi packets would ever arrive.

Why do we need `configurator`?

Real networks require IP addresses for routing. The `Ipv4NetworkConfigurator` automatically assigns IP addresses to all hosts and builds routing tables, so you do not have to configure them manually. Setting `assignDisjunctSubnetAddresses = false` places all hosts on the **same subnet**, so they can communicate directly without extra routing.

4.6 Connections: Wiring the Devices Together

The `connections` block defines the physical **cables** between devices.

```
connections:
    // Syntax: deviceA.gate++ <--> ChannelType <--> deviceB.gate
   ++;
    accessPoint.ethg++ <--> Eth100M <--> router.ethg++;
    wiredHost1.ethg++  <--> Eth100M <--> accessPoint.ethg++;
    wiredHost2.ethg++  <--> Eth100M <--> router.ethg++;
```

Listing 6: Connections block

4.6.1 Understanding the Connection Syntax

Symbol	Meaning
`deviceA.ethg`	The Ethernet gate (port) of `deviceA`
`++`	Automatically use the next available port number
`<-->`	Bidirectional connection (data flows both ways)
`Eth100M`	The channel type: a 100 Mbps Ethernet cable

Table 2: Connection syntax reference

What about wireless connections?

Wireless connections are **never written in the connections block**. A `WirelessHost` automatically associates with the nearest `AccessPoint` through the shared `radioMedium` module at runtime. You only declare wired cables in `connections`.

4.7 A Complete NED File Example

Below is the complete NED file for the `WiredAndWirelessHostsWithAP` network, which you will use as your reference throughout this course.

```
//
// SPDX-License-Identifier: LGPL-3.0-or-later
//

package inet.examples.wireless.wiredandwirelesshostswithap;

import inet.networklayer.configurator.ipv4.Ipv4NetworkConfigurator
   ;
import inet.node.ethernet.Eth100M;
import inet.node.inet.Router;
import inet.node.inet.StandardHost;
import inet.node.inet.WirelessHost;
import inet.node.wireless.AccessPoint;
import inet.physicallayer.wireless.ieee80211.packetlevel.
   Ieee80211ScalarRadioMedium;
import inet.visualizer.contract.IIntegratedVisualizer;
```

```
15
16 network WiredAndWirelessHostsWithAP
17 {
18     submodules:
19         visualizer: <default(firstAvailableOrEmpty("
   IntegratedCanvasVisualizer"))>
20                     like IIntegratedVisualizer if typename != "" {
21             parameters:
22                 @display("p=100,300;is=s");
23         }
24         configurator: Ipv4NetworkConfigurator {
25             parameters:
26                 assignDisjunctSubnetAddresses = false;
27                 @display("p=100,100;is=s");
28         }
29         radioMedium: Ieee80211ScalarRadioMedium {
30             parameters:
31                 @display("p=100,200;is=s");
32         }
33         wirelessHost1: WirelessHost {
34             parameters:
35                 @display("p=62,69");
36         }
37         wiredHost1: StandardHost {
38             parameters:
39                 @display("p=296,134");
40         }
41         wiredHost2: StandardHost {
42             parameters:
43                 @display("p=412,70");
44         }
45         router: Router {
46             parameters:
47                 @display("p=296,69");
48         }
49         accessPoint: AccessPoint {
50             parameters:
51                 @display("p=205,69");
52         }
53     connections:
54         accessPoint.ethg++ <--> Eth100M <--> router.ethg++;
55         wiredHost1.ethg++  <--> Eth100M <--> accessPoint.ethg++;
56         wiredHost2.ethg++  <--> Eth100M <--> router.ethg++;
57 }
```

Listing 7: Complete NED file: WiredAndWirelessHostsWithAP.ned

5. Understanding INI Files

The `omnetpp.ini` file is where you configure **how the simulation behaves**. While the NED file answers *"what exists?"*, the INI file answers *"what does it do, and how?"*

5.1 Sections in the INI File

An INI file is divided into **sections**, each starting with a name in square brackets.

```
[General]
# Settings here apply to ALL configurations
network = WiredAndWirelessHostsWithAP
sim-time-limit = 400s

[Config Experiment1]
# Settings here apply only to "Experiment1"
# They override or extend [General] settings
```

Listing 8: INI file section structure

The `[General]` section is special — it is always active. Named `[Config ...]` sections are selected by the user at runtime when launching the simulation.

5.2 Wildcard Patterns: Targeting Modules

INI file parameters use **wildcard patterns** to target modules in the network hierarchy.

Pattern	**What it matches**
`**`	Any module at any depth in the hierarchy
`*`	Any single module name at one level
`*Host*`	Any module whose name *contains* "Host"
`wiredHost1`	Only the module named exactly `wiredHost1`
`app[0]`	Only application index 0
`app[*]`	All application indices
`app[1..]`	Application index 1 and above

Table 3: Wildcard pattern reference

Specificity Rule — Very Important!

When two INI lines configure the same parameter, **the more specific pattern always wins**, regardless of the order in the file.

Example:

```
**.*Host*.app[0].typename = "UdpEchoApp"    <-
more specific (wins for app[0])
**.*Host*.app[*].typename = "UdpBasicApp"   <-
less specific (applies to app[1], app[2], ...)
```

`app[0]` is an exact index and is more specific than the wildcard `app[*]`, so `app[0]` keeps `UdpEchoApp` even though `app[*]` also targets it. Order in the file does **not** matter.

5.3 Key INI Parameters Explained

5.3.1 Global Simulation Settings

```
[General]
network = WiredAndWirelessHostsWithAP   // Which NED network to
   load
sim-time-limit = 400s                   // Stop simulation at 400
   simulated seconds
**.addDefaultRoutes = false             // Do not auto-add default
    gateway routes
```

Listing 9: Global settings in omnetpp.ini

`sim-time-limit` is simulated time, not real clock time. A 400-second simulation may finish in just a few real seconds on modern hardware.

`addDefaultRoutes = false` disables the automatic catch-all gateway route. In a small closed network where every host already has a specific route to every other host, a default route is unnecessary. Hosts can still communicate because the configurator adds **direct routes** for all known destinations.

5.3.2 Configuring the Number of Applications

```
**.*Host*.numApps = 3
```

Listing 10: Setting the number of apps per host

This gives every host (whose name contains "Host") exactly **3 application slots**: `app[0]`, `app[1]`, and `app[2]`. Think of these as three programs running simultaneously on each computer.

5.3.3 Configuring Application Types

```
// app[0] on every host: a UDP echo server (reflects packets back)
**.*Host*.app[0].typename = "UdpEchoApp"
**.*Host*.app[0].localPort = 1000

// app[1] and app[2] on every host: UDP traffic senders
// (app[*] acts as default; app[0] keeps UdpEchoApp due to
   specificity rule)
**.*Host*.app[*].typename = "UdpBasicApp"
```

Listing 11: Setting application types

UdpEchoApp vs UdpBasicApp

UdpEchoApp — Listens on a port and immediately sends back any packet it receives. It is the server side.

UdpBasicApp — Generates and sends UDP packets to a specified destination at regular intervals. It is the client/sender side.

5.3.4 Configuring Traffic Destinations

```
**.[wiredHost1.app][1].destAddresses  = "wirelessHost1"
**.[wiredHost1.app][2].destAddresses  = "wiredHost2"
**.[wiredHost2.app][1].destAddresses  = "wirelessHost1"
**.[wiredHost2.app][2].destAddresses  = "wiredHost1"
**.[wirelessHost1.app][1].destAddresses = "wiredHost1"
**.[wirelessHost1.app][2].destAddresses = "wiredHost2"
```

Listing 12: Setting destination addresses for each app

Each host's two sending apps are directed to the other two hosts. The destination is specified by the **module name** as defined in the NED file. OMNeT++ resolves this name to the host's IP address automatically at runtime.

5.3.5 Configuring Traffic Parameters

```
// All parameters below apply to app[1] and app[2] (index 1 and
   above)
**.*Host*.app[1..].destPort      = 1000    // Send to port 1000 (
   UdpEchoApp listens here)
**.*Host*.app[1..].messageLength = 100B    // Each packet is 100
   bytes
**.*Host*.app[1..].sendInterval  = 1s      // Send one packet
   every 1 second
**.*Host*.app[1..].stopTime      = 300s    // Stop sending at 300
   simulated seconds
```

Listing 13: Traffic parameters for all sending apps

5.3.6 Other Useful Parameters

```
**.initialZ = 0m         // Place all nodes at height 0 (flat
   ground, 3D visualizer)

**.vector-recording = false              // Disable time-series
   result recording globally
**.wiredHost1.*.vector-recording = true // Re-enable only for
   wiredHost1

// Random traffic: exponential inter-packet interval with mean 1s
**.*Host*.app[1..].sendInterval = exponential(1s)

// Staggered start times
**.[wiredHost1.app][1].startTime  = 0s
**.[wiredHost2.app][1].startTime  = 5s
**.[wirelessHost1.app][1].startTime = 10s
```

Listing 14: Additional INI parameters

5.4 A Complete INI File Example

```
[General]
network = WiredAndWirelessHostsWithAP
sim-time-limit = 400s

**.addDefaultRoutes = false

**.*Host*.numApps = 3

**.*Host*.app[0].typename  = "UdpEchoApp"
**.*Host*.app[0].localPort = 1000

**.*Host*.app[*].typename  = "UdpBasicApp"

**.[wiredHost1.app][1].destAddresses   = "wirelessHost1"
**.[wiredHost1.app][2].destAddresses   = "wiredHost2"
**.[wiredHost2.app][1].destAddresses   = "wirelessHost1"
**.[wiredHost2.app][2].destAddresses   = "wiredHost1"
**.[wirelessHost1.app][1].destAddresses = "wiredHost1"
**.[wirelessHost1.app][2].destAddresses = "wiredHost2"

**.*Host*.app[1..].destPort      = 1000
**.*Host*.app[1..].messageLength = 100B
**.*Host*.app[1..].sendInterval  = 1s
**.*Host*.app[1..].stopTime      = 300s

**.initialZ = 0m
```

Listing 15: Complete omnetpp.ini

6. Creating a New Network in the Same Folder

Now that you understand NED and INI files, here is how to create your own network inside the same INET example folder.

6.1 Step 1 — Create a New NED File

1. In the Project Explorer, navigate to:
   `inet/examples/wireless/wiredandwirelesshostswithap/`

2. Right-click the folder → **New** → **Network Description File (NED)**.

3. Name the file (e.g., `MyNetwork.ned`) and click **Finish**.

4. The IDE creates the file with a skeleton. Make sure the `package` line at the top reads:
   `package inet.examples.wireless.wiredandwirelesshostswithap;`

6.2 Step 2 — Write the Network Definition

Add your imports and `network` block. Here is a minimal example — a simple network with two wired hosts and a router:

```
package inet.examples.wireless.wiredandwirelesshostswithap;

import inet.networklayer.configurator.ipv4.Ipv4NetworkConfigurator
    ;
import inet.node.ethernet.Eth100M;
import inet.node.inet.Router;
import inet.node.inet.StandardHost;

network MyNetwork
{
    submodules:
        configurator: Ipv4NetworkConfigurator {
            parameters:
                assignDisjunctSubnetAddresses = false;
                @display("p=100,100;is=s");
        }
        hostA: StandardHost {
            parameters:
                @display("p=100,250");
        }
        hostB: StandardHost {
            parameters:
                @display("p=400,250");
        }
        router: Router {
            parameters:
                @display("p=250,250");
        }
    connections:
        hostA.ethg++  <--> Eth100M <--> router.ethg++;
        hostB.ethg++  <--> Eth100M <--> router.ethg++;
}
```

Listing 16: MyNetwork.ned — a simple two-host network

6.3 Step 3 — Add a Configuration Section to omnetpp.ini

Open the existing `omnetpp.ini` file in the same folder and add a new `[Config ...]` section at the bottom:

```
[Config MyNetworkConfig]
network = MyNetwork
sim-time-limit = 100s

**.*Host*.numApps = 2

**.*Host*.app[0].typename  = "UdpEchoApp"
**.*Host*.app[0].localPort = 1000

**.*Host*.app[*].typename  = "UdpBasicApp"

```

```
**.hostA.app[1].destAddresses = "hostB"
**.hostB.app[1].destAddresses = "hostA"

**.*Host*.app[1].destPort      = 1000
**.*Host*.app[1].messageLength = 100B
**.*Host*.app[1].sendInterval  = 1s
**.*Host*.app[1].stopTime      = 80s
```

Listing 17: Adding MyNetwork configuration to omnetpp.ini

6.4 Step 4 — Run the Simulation

1. Right-click `omnetpp.ini` → **Run As** → **OMNeT++ Simulation**.
2. A dialog appears listing all available configurations. Select **MyNetworkConfig** and click **OK**.
3. Qtenv opens and displays your network canvas.
4. Press the green **Play** button to start the simulation.
5. Watch the animated packets travel between `hostA` and `hostB` via the router.
6. When the simulation ends (at $t = 100\,\text{s}$), check the **Console** tab for any output messages.

Tip

You can slow down the simulation in Qtenv using the speed slider in the toolbar. Dragging it to the left makes packets animate more slowly so you can observe them clearly. Dragging to the right speeds things up for long simulations.

7. Running the Simulation: Qtenv Controls

7.1 Main Control Buttons

7.2 Inspecting Modules

Click on any module icon on the canvas to open its **inspector panel**. You can examine:

- The module's current parameter values
- Its routing table (for hosts and routers)
- Statistics such as packets sent, received, and dropped
- The contents of its message queue

Double-clicking a host opens its internal compound module view, showing all sub-components (NIC, IP layer, transport layer, applications).

Button / Action	What it does
Play (Run)	Runs the simulation continuously until `sim-time-limit` or until you pause it
Step	Advances exactly one event, then pauses. Useful for detailed inspection
Pause	Pauses a running simulation at the current simulated time
Stop	Stops and resets the simulation
Run Until...	Runs until a specific simulated time that you enter (**Simulate** → **Run Until**)
Fast	Runs without packet animation (much faster for long simulations)

Table 4: Qtenv control reference

7.3 Reading the Console Output

Application modules in INET print status messages to the OMNeT++ console. For example, a `UdpBasicApp` logs every packet it sends, and `UdpEchoApp` logs every packet it echoes. These messages are visible in the **Console** tab at the bottom of the IDE during or after a run.

8. Common Errors and How to Fix Them

9. Quick Reference Summary

9.1 NED File Checklist

☐ `package` line matches the folder path exactly

☐ All used module types are imported

☐ `Ipv4NetworkConfigurator` is declared as a submodule

☐ `Ieee80211ScalarRadioMedium` declared if any wireless hosts are used

☐ All wired connections written in the `connections` block

☐ Network name matches what is referenced in `omnetpp.ini`

9.2 INI File Checklist

☐ `network` = matches the NED network name exactly

☐ `sim-time-limit` is set

☐ `numApps` is set for all hosts

Error / Symptom	Likely Cause and Fix
Red underline in NED file	Syntax error or wrong module name. Check spelling of module types and that all imports are present.
`Network not found` at runtime	The `network =` value in omnetpp.ini does not match any network name in your NED files. Check capitalisation.
`Package mismatch` error	The `package` line in the NED file does not match the folder path. Correct the package declaration.
No packets visible in Qtenv	Check that `startTime < stopTime < sim-time-limit`. Also verify `destAddresses` is not empty.
Hosts cannot communicate	Ensure `Ipv4NetworkConfigurator` is present as a submodule in the NED file. Without it, no IP addresses are assigned.
`Cannot connect gate` error	You tried to connect a gate type that the module does not have (e.g., connecting `ethg` to a wireless-only host). Check module documentation for available gates.
Wireless host not receiving	`radioMedium` submodule is missing from the NED file, or the wireless host is out of radio range of the access point.

Table 5: Common errors and fixes

☐ `typename` is set for each app index

☐ `destAddresses` is set for all `UdpBasicApp` instances

☐ `destPort` matches the `localPort` of the receiving `UdpEchoApp`

☐ Units are included for all values (`B`, `s`, `Mbps`)

9.3 Key Terminology Glossary

Term	**Meaning**
Discrete-event simulation	Time advances in jumps from one event to the next
NED file	Network description file defining topology
INI file	Configuration file defining simulation behaviour
Submodule	A device or component declared inside a network
Gate	A communication port on a module (`ethg`, `pppg`)
Channel	A link connecting two gates (e.g., `Eth100M`)
Package	Folder-based namespace for NED files
Qtenv	Graphical runtime window for running and visualising simulations
UdpEchoApp	An app that echoes received packets back to the sender
UdpBasicApp	An app that generates and sends UDP packets at regular intervals
radioMedium	The simulated wireless propagation medium ("air")
configurator	Module that auto-assigns IP addresses and routing tables
Wildcard (**)	Matches any module at any depth in the hierarchy
Specificity	More precise INI patterns override less precise ones

Table 6: Key terminology glossary